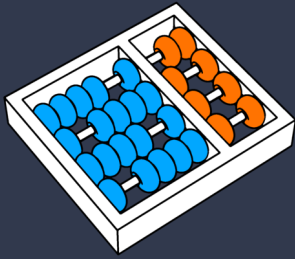




UNICAMP



Deep Agents

Prof. Dr. Julio Cesar dos Reis
dosreis@unicamp.br

Agenda

- ❑ Motivação
- ❑ Definição
- ❑ Componentes
- ❑ Modo de Uso
- ❑ Exemplo
- ❑ Benefícios e Limitações

Motivação

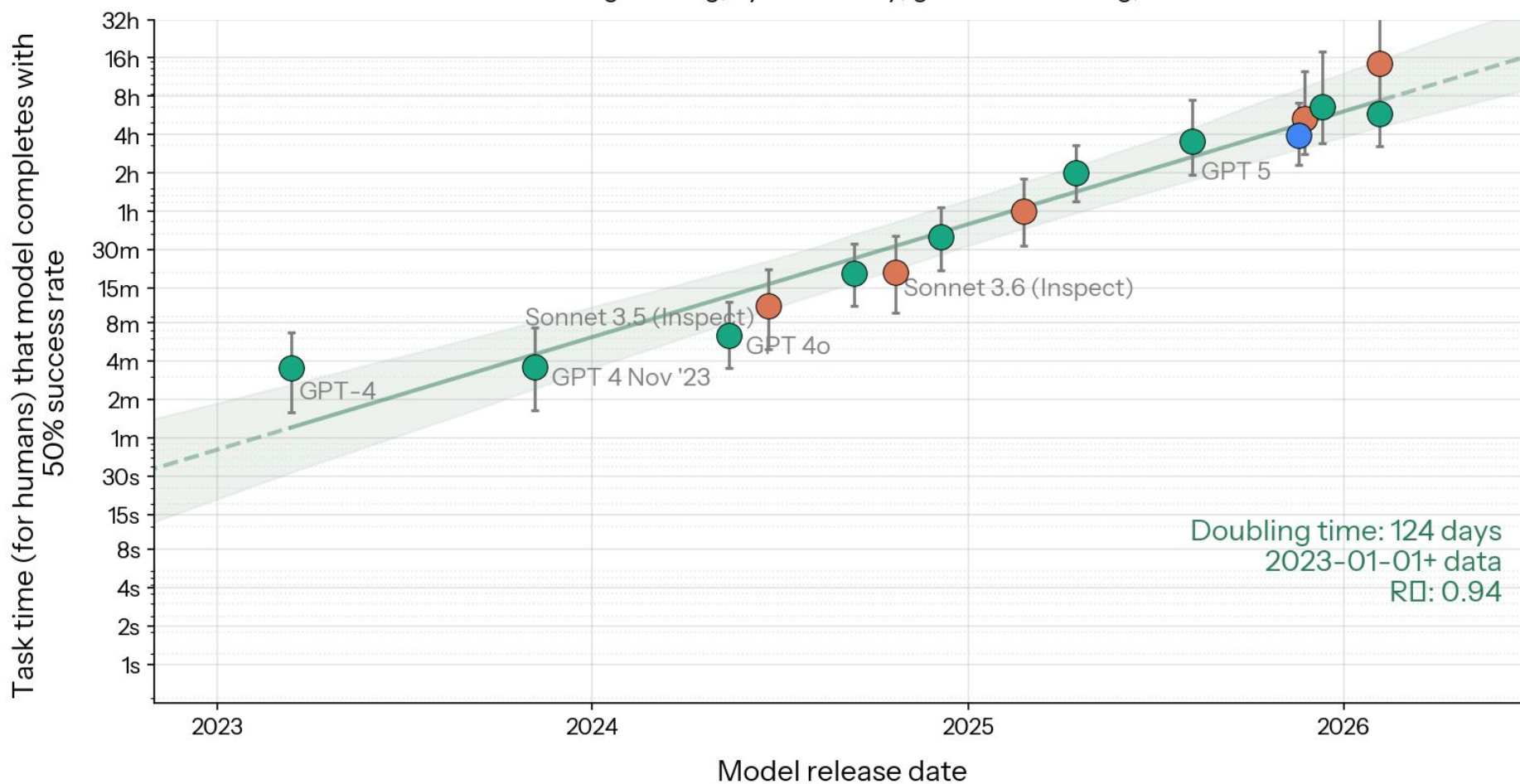
- ❑ Agentes trabalhando em “tarefas gerais e complexas” e em “tarefas de longo horizonte” temporal.
- ❑ Tarefas gerais: em vez de resolver um problema específico e bem delimitado, o agente lida com objetivos abertos, em domínios variados, sem um fluxo pré-definido.
- ❑ Tarefas de longo horizonte: Envolvem muitas etapas encadeadas ao longo de minutos, horas ou dias.

Tarefa de longo horizonte (Exemplo)

- ❑ Objetivo: “produzir uma revisão bibliográfica sobre *Agentic AI*, com 20 fontes, organizada por tema e com lacunas de pesquisa identificadas”
- ❑ Desafios:
 - Dezenas de buscas (fontes de dados) e leituras de artigos
 - Resultados longos que não cabem simultaneamente no contexto
 - Necessário organizar diversos tipos de contextos
 - Critérios de inclusão que evoluem conforme a leitura avança
 - Um erro de escopo no início compromete todo o resultado final

Motivação

Length of tasks AI agents have been able to complete autonomously
for 228 software engineering, cybersecurity, general reasoning, and ML tasks



Motivação

- ❑ Os LLMs ficaram muito bons em resolver tarefas pequenas.
- ❑ Agora queremos que eles resolvam tarefas que duram:
 - dezenas de minutos
 - horas
 - dias
- ❑ Isso muda completamente a arquitetura.

O Agente "Raso" (*Shallow Agent*)

- ❑ Modelo decide a cada turno: chamar uma ferramenta ou responder. Sem plano explícito, sem memória externa, sem delegação.
- ❑ **Loop:** pensar → agir → observar → repetir

Problemas com o Agente "Raso"

Sem planejamento

→ Perde o objetivo no meio
refaz ou pula etapas

Sem memória externa

→ Estoura contexto
o que importava rola para fora da janela

Sem decomposição

→ Mistura tudo

Sem arquivos

→ Reprocessa informação

Sem especialização

→ Mesmo agente faz tudo

Sem delegação

→ Contexto enorme
Detalhes de subtarefas poluem o contexto principal

Motivação

Antes

Pergunta



Resposta

Agora

1

Objetivo

2

Planejar

3

Pesquisar

4

Executar

5

Corrigir

6

Delegar

7

Produzir Resultado

O crescimento das tarefas

- ❏ Resolver um projeto

- ❏ Exemplos:

- desenvolver software
- revisar literatura
- criar um plano de negócios
- migrar um sistema
- escrever um artigo científico
- analisar milhares de documentos

Exemplo

- ❑ Produzir uma revisão sistemática
- ❑ Não é apenas chamar uma ferramenta.

- ❑ É necessário:
 - definir estratégia
 - buscar artigos
 - remover duplicatas
 - avaliar qualidade
 - resumir
 - reorganizar
 - identificar lacunas
 - produzir texto final

Deep Agents

Deep Agents

- ❑ São arquiteturas de agentes projetadas para executar tarefas abertas, complexas e de longo horizonte temporal
- ❑ Integra planejamento, gerenciamento de contexto, arquivos, skills e delegação de subtarefas em um único ciclo de execução.
- ❑ Não é um novo modelo.
- ❑ Não é um novo LLM.
- ❑ **É uma arquitetura.**

Deep Agents

❏ Um *agent harness*

- Um Harness é um ambiente de execução que coordena automaticamente o ciclo de vida do agente
 - Um arcabouço que reúne em um único ciclo de execução capacidades como planejamento, arquivos, subagentes e skills

❏ Inspirado em ferramentas como Claude Code e Deep Research.

Harness de Agentes



Harness

Um ambiente de execução que coordena automaticamente o ciclo de vida de deep agents.

Ele une planejamento, gerenciamento de contexto e ferramentas em um único ciclo de execução coeso.



Planner

Decompõe tarefas em planos estruturados e prontos para execução.



Subagentes

Delega subtarefas complexas a unidades de execução especializadas e isoladas.



Arquivos

Gerencia a persistência de dados e o estado de execução fora do contexto do LLM.



Ferramentas

Integrações externas, APIs e motores de execução em sandbox.



Skills

Capacidades especializadas e estratégias de execução reutilizáveis.

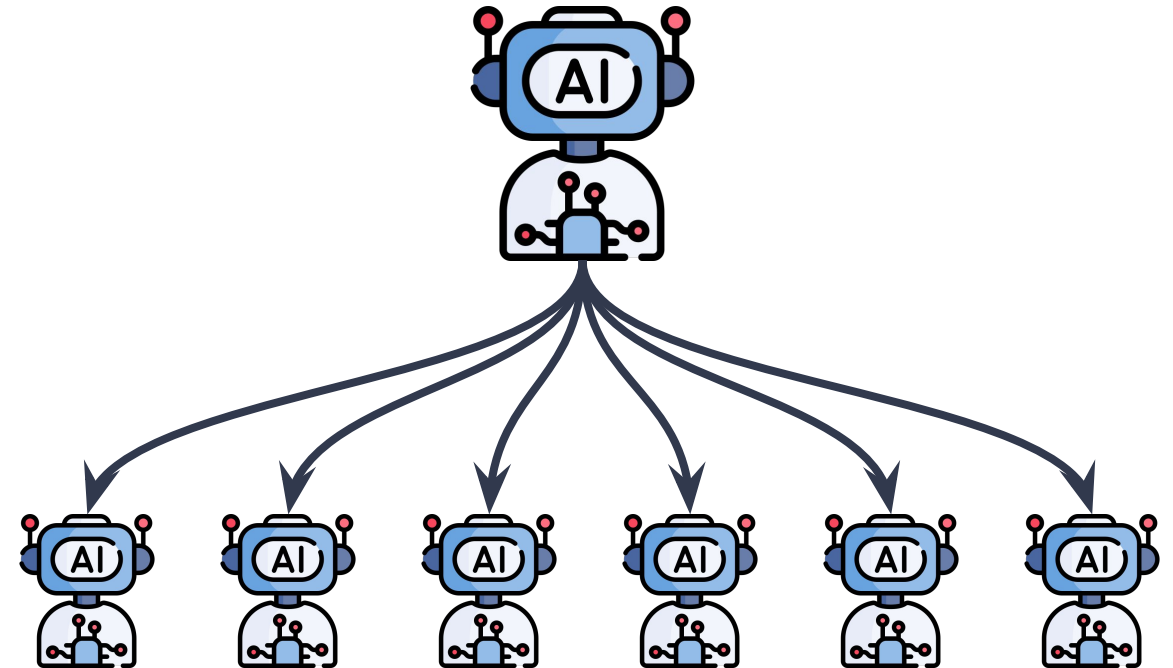


LLM

O modelo de fundação central que atua como o processador cognitivo.

Deep Agents

- ❑ **Decompõe explicitamente** uma tarefa longa em um plano
- ❑ **Mantém estado** fora da janela de contexto
- ❑ **Delega subtarefas** a subagentes



Agente Raso e Deep Agent

Aspecto	Agente Raso	Agente Profundo (Deep Agent)
Plano	Implícito no histórico	Explícito e revisável
Memória	Janela de contexto	Externa e persistente
Subtarefas	No mesmo contexto	Delegadas com contexto isolado
Instruções	Prompt curto	Prompt extenso e procedimental
Melhor caso	Tarefas de poucos passos	Tarefas de longo horizonte

Histórico

- ❑ Termo popularizado pela LangChain com a biblioteca deepagents
- ❑ Declaram inspiração direta no Claude Code
- ❑ **O padrão, porém, já existia:** Claude Code, Deep Research e Manus

Engenharia de Contexto

- ❑ **Em tarefas longas** a janela de contexto é um recurso finito
- ❑ Cada observação, cada resultado de ferramenta e cada passo do raciocínio consome “orçamento”
 - Gerenciar esse “orçamento” passa a ser o problema central da arquitetura
- ❑ **Componentes do *Deep Agent* definidos como resposta a esse problema**

Estratégias de contexto

- ❑ Cada componente do Deep Agent implementa uma dessas estratégias.

Estratégia	Ideia	Componente
<i>Escrever</i>	Gravar fora da janela de contexto	Sistema de arquivos
<i>Selecionar</i>	Trazer de volta apenas o relevante	Sistema de arquivos
<i>Comprimir</i>	Resumir o que já foi feito	Planejamento / sumarização
<i>Isolar</i>	Separar contextos por responsabilidade	Subagentes

Componentes

Componentes do Deep Agents

- ❑ Mantêm o mesmo loop de tool calling
- ❑ A mudança está na estrutura em volta
- ❑ Quatro componentes fazem esse trabalho

Componentes do Deep Agents



Componentes

- ❑ **Prompt de sistema:** Instruções extensas, com exemplos e regras de uso das ferramentas.
- ❑ **Planejamento:** O agente escreve e atualiza sua própria lista de tarefas.
- ❑ **Sistema de arquivos:** Memória fora da janela de contexto: resultados grandes e conhecimento que persiste entre sessões.
- ❑ **Subagentes:** Sub tarefas delegadas a agentes com contexto isolado.

Componentes do Deep Agents



Prompt de Sistema

- ❑ Não é apenas sobre papel do agente ("você é um assistente útil")
- ❑ **É um documento procedimental:** quando usar cada ferramenta, em que ordem, o que fazer em caso de erro
- ❑ Define como o agente trabalha, e não apenas o que ele é
- ❑ Ex: No *Claude Code*, o prompt de sistema tem milhares de tokens

Prompt de Sistema e Skills

- ❑ Relacionado às Skills
- ❑ Skills empacotam esse conhecimento de forma modular, versionada e sob demanda
- ❑ A biblioteca ***deepagents*** incorporou skills como primitiva do *harness*
- ❑ **Tools: o que posso fazer. MCP: onde posso acessar. Skills: como devo fazer.**
- ❑ O prompt de sistema é o veículo do "como"

Componentes do Deep Agents



Planejamento

- ❑ O agente recebe uma ferramenta para escrever e atualizar sua lista de tarefas.
- ❑ O plano deixa de ser implícito no histórico e passa a ser um artefato explícito
- ❑ A cada passo o agente relê o plano, marca o que concluiu (check) e ajusta o que falta
- ❑ **Combate direto à perda de objetivo em tarefas longas**

Planejamento

- ❑ **Planejamento estático:** um plano único sem revisão
- ❑ **Planejamento dinâmico:** replaneja a cada nova observação
- ❑ ***Plan-and-Execute*:** planner, executor e replanner como componentes separados

- ❑ **No Deep Agent o planejamento não é um nó do grafo**
 - é uma ferramenta que o próprio agente decide chamar
 - Isso o aproxima do planejamento dinâmico, mas sob controle do modelo, não do designer

Componentes do Deep Agents



Sistema de Arquivos Virtual

- ❑ O agente pode ler e escrever arquivos por meio de ferramentas
- ❑ Resultados grandes vão para o arquivo
- ❑ Apenas a referência permanece no contexto
- ❑ É a estratégia "escrever" e "selecionar" em funcionamento

- ❑ **Backends possíveis:** memória, disco local, store do LangGraph

Descarregamento de contexto (*offloading*)

- ❑ **Sem arquivos:** cada resultado de busca permanece inteiro no histórico até estourar a janela de contexto
- ❑ **Com arquivos**
 - o resultado é gravado
 - o contexto guarda apenas o caminho e um resumo
- ❑ O agente relê sob demanda, quando aquele conteúdo volta a ser necessário
- ❑ **O contexto deixa de ser um acumulador**
- ❑ **Passa a ser um espaço de trabalho**

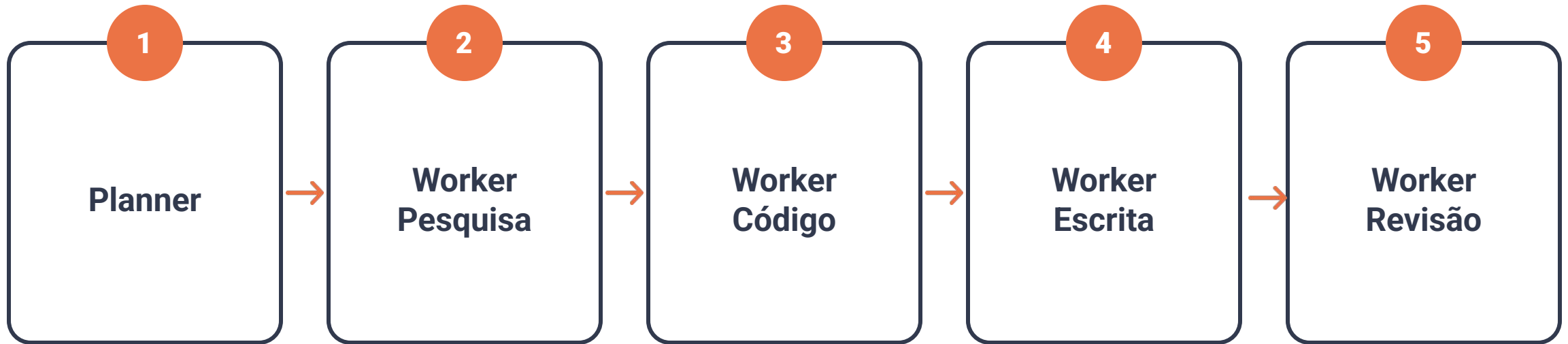
Componentes do Deep Agents



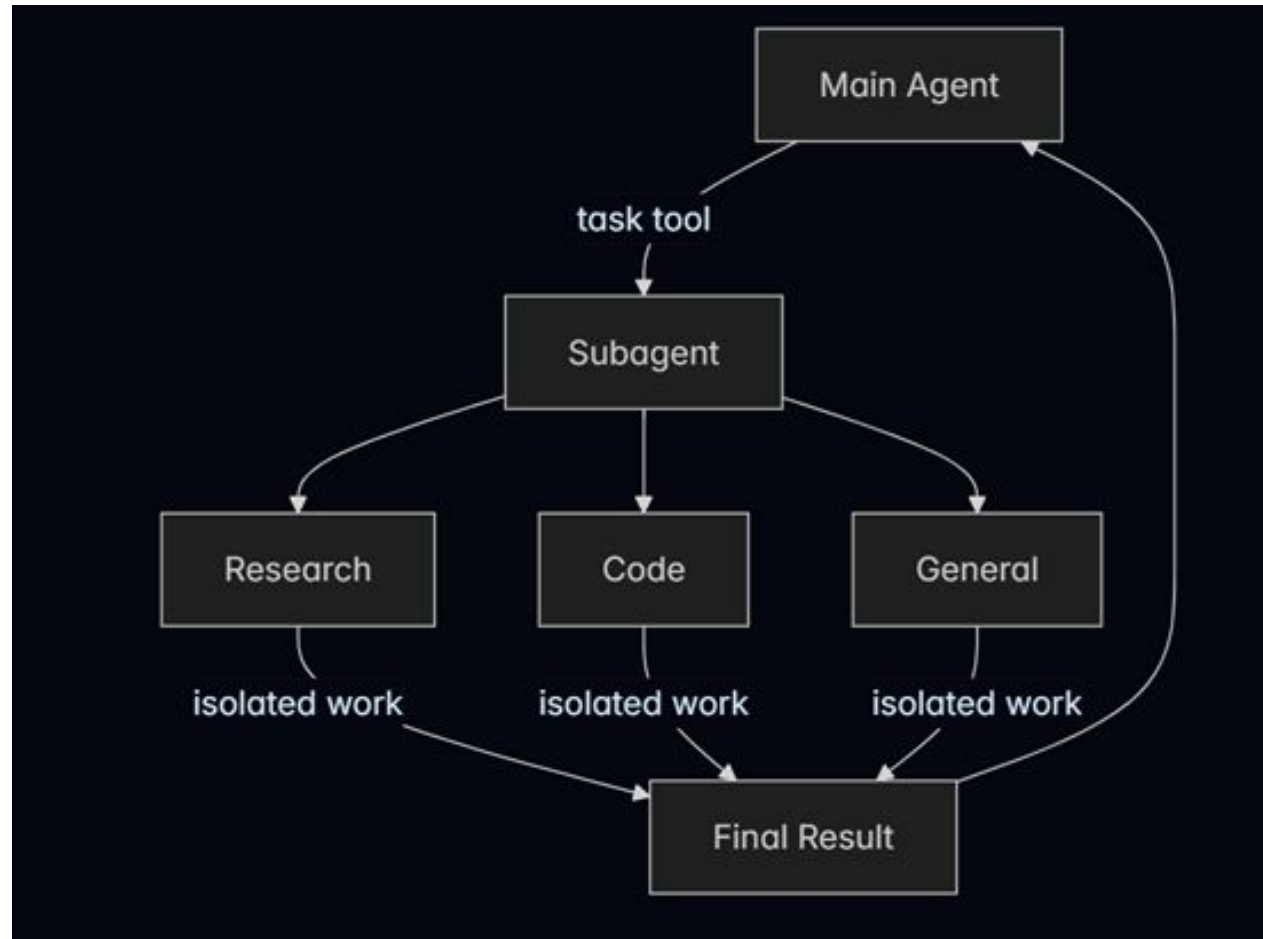
Subagentes

- ❑ O agente principal **delega** uma subtarefa a outro agente (subagente)
- ❑ O subagente executa em contexto próprio e devolve apenas o resultado
- ❑ Os passos intermediários não entram no contexto principal
- ❑ **A Estratégia é "isolar"**: separar contextos por responsabilidade
 - Cada agente recebe contexto reduzido.
 - Depois devolve apenas o resultado.

Workflow de Subagentes



Workflow de Subagentes

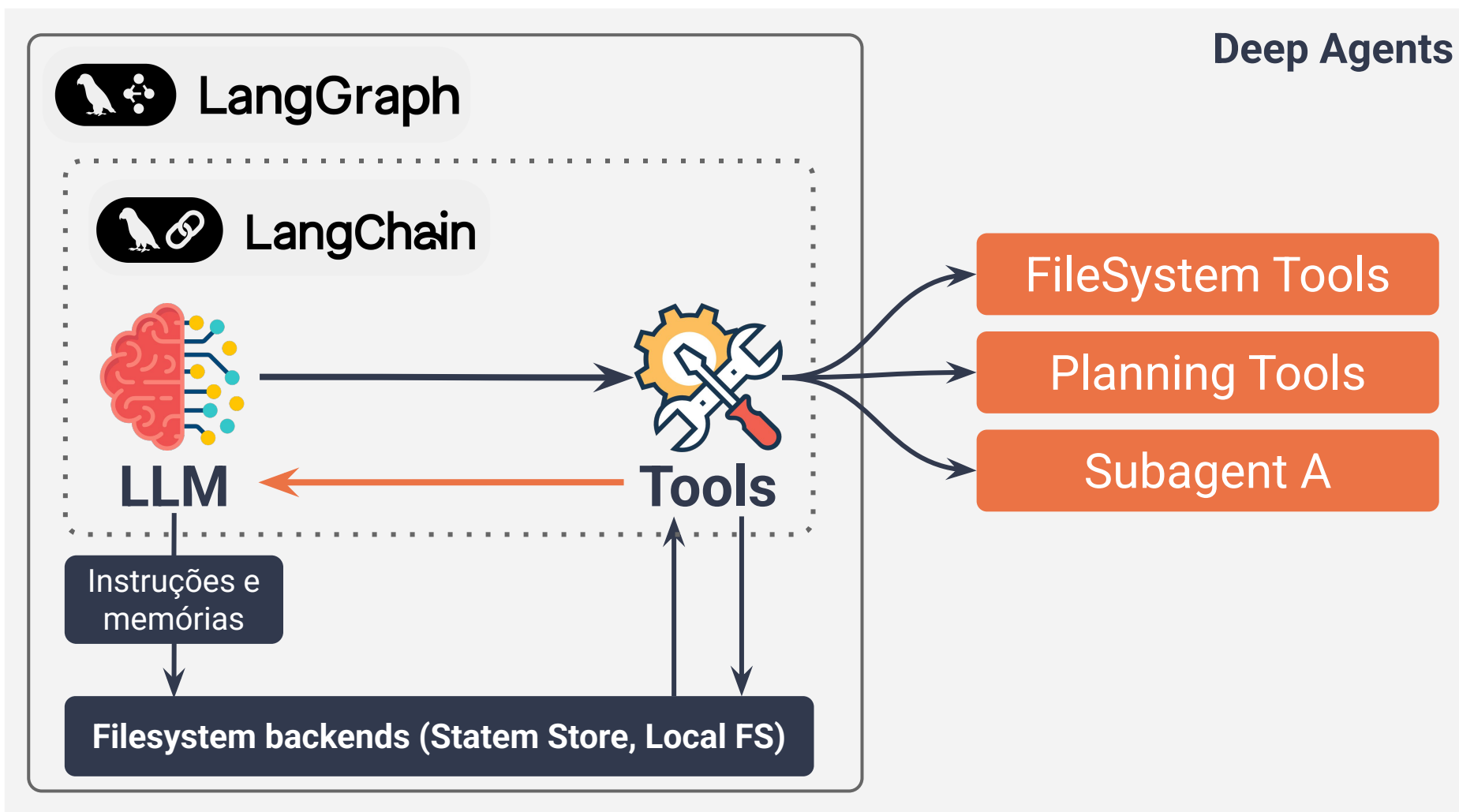


Fluxo Completo Iterativo de Execução



Deep Agents no LangChain

Deep Agents no LangGraph/LangChain



O harness em código

```
from deepagents import create_deep_agent

agent = create_deep_agent(
    model=MODELO,
    tools=[buscar_artigos, ler_artigo],
    system_prompt=PROMPT_DE_PESQUISA)

result = agent.invoke(
    {"messages": "Revise a literatura sobre Agentic AI" })
```

LOOP INTEGRADO

O harness já acopla tudo ao mesmo ciclo de execução:

Planejamento

Criação de planos dinâmicos

Arquivos

Persistência e leitura direta

Subagentes

Delegação de tarefas específicas

O harness em código



Traço de execução (1/4)

PLANO

1. Definir critérios de inclusão
2. Buscar fontes (20)
3. Ler e fichar cada fonte
4. Agrupar por tema
5. Redigir a síntese

ARQUIVOS: (vazio)

SUBAGENTES: nenhum

Traço de execução (2/4)

PLANO

1. Definir critérios de inclusão [ok]
2. Buscar fontes (20) [ok]
3. Ler e fichar cada fonte
4. Agrupar por tema
5. Redigir a síntese

ARQUIVOS: [critérios.md](#), [fontes.md](#)

SUBAGENTES: nenhum

Traço de execução (3/4)

PLANO

1. Definir critérios de inclusão [ok]
2. Buscar fontes (20) [ok]
3. Ler e fichar cada fonte <-- delegado
4. Agrupar por tema
5. Redigir a síntese

ARQUIVOS: [critérios.md](#), [fontes.md](#)

SUBAGENTES: [leitor \(x20\)](#)

Cada subagente devolve apenas a ficha, nunca o texto integral do artigo (isolamento de contexto).

Traço de execução (4/4)

PLANO

1. Definir critérios de inclusão [ok]
2. Buscar fontes (20) [ok]
3. Ler e fichar cada fonte [ok]
4. Agrupar por tema [ok]
5. Redigir a síntese [ok]
6. Identificar lacunas <-- delegado

ARQUIVOS: + temas.md, sintese.md

SUBAGENTES: leitor (x20)

O plano mudou porque a execução trouxe informação nova.

Prompt de planejamento

"Antes de agir, escreva a lista de tarefas com *write_todos*."

"Marque cada item como concluído imediatamente após concluí-lo."

"Se a execução revelar informação que altere o plano, reescreva a lista."

"Nunca marque como concluída uma tarefa que não foi executada."

A última instrução existe porque essa é uma falha real e frequente.

Benefícios e Limitações

Benefícios

- ❑ Sustentação de tarefas longas sem perda de objetivo e de contexto
- ❑ Contexto controlado, independentemente do volume de dados intermediários
- ❑ Progresso auditável: o plano é um artefato inspecionável
- ❑ Recuperação de erro: é possível revisar o plano sem reiniciar a tarefa
- ❑ Reuso: o mesmo *harness* serve a domínios diferentes

Benefícios

Tradicional	Deep Agent
contexto cresce continuamente	contexto controlado
sem planejamento	planejamento
sem arquivos	offloading
um agente	vários especialistas
prompt enorme	skills
pouca escalabilidade	longo horizonte

Desafios

- ❑ O agente pode marcar uma tarefa como concluída sem tê-la executado
- ❑ Subagentes podem devolver resumos que omitem o essencial
 - geram resultados inadequados
- ❑ Depuração é difícil: execuções longas e não determinísticas
- ❑ Custo cresce de forma pouco previsível
- ❑ Um plano ruim conduz a execução inteira para o lugar errado
- ❑ Indicados para fluxos complexos, mas exigem boa governança e observabilidade.

Limitações

- ❑ custo elevado
- ❑ maior latência
- ❑ debugging difícil
- ❑ observabilidade
- ❑ coordenação
- ❑ consistência entre subagentes
- ❑ gerenciamento de arquivos
- ❑ dependência de boas skills

Quando utilizar

Tarefa	Deep Agent?
FAQ	Não
Chatbot simples	Não
Programação	Sim
Pesquisa	Sim
Auditoria	Sim
Revisão sistemática	Sim
Engenharia de software	Sim
Data Science	Sim

Síntese

- ❑ Tarefas mais gerais (longas e complexas) expõem os limites do agente
- ❑ Deep Agents surgiram para resolver tarefas abertas e de longo horizonte temporal
 - **Não** são um agente mais inteligente
 - Estrutura para tratar tarefas complexas e longas evitando a degradação
 - Uma arquitetura que organiza a execução de agentes
- ❑ O Harness coordena planejamento, execução e contexto.
- ❑ Subagentes permitem divisão e paralelização de tarefas.
- ❑ Memória e arquivos ampliam a capacidade operacional do agente.

Referências

LANGCHAIN. Deep Agents - Documentação oficial.

docs.langchain.com/oss/python/deepagents

<https://www.langchain.com/deep-agents>

LANGCHAIN. Repositório deepagents.

<https://github.com/langchain-ai/deepagents>

<https://github.com/langchain-ai/deepagents-quickstarts>

<https://github.com/langchain-ai/deepagents/tree/main/examples>

METR. Measuring AI Ability to Complete Long Tasks. 2025.